



Parking Finder

Project Engineering

Year 4

Sean Maloney

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Sean Maloney

AI Declaration

- Generative AI tools were used as support tools during the preparation of this project.
- AI assistance, where used, was used for tasks such as brainstorming, language refinement, formatting support, structure planning, debugging, generation of code, rewriting of code, and web scraping information not easily found.
- All technical design, implementation, testing, evaluation, and final written content were reviewed, validated, and approved by the author.
- The author takes full responsibility for the accuracy, originality, and academic integrity of this submission.

Acknowledgements

Fergus Downey – For assisting in the project whenever possible

Johanan Seeruthen – Helped develop the camera script for raspberry pi

Tamer Zraiq – Offered helpful advice for project planning

David Jayakumar - Offered helpful advice for using GitHub professionally

Table of Contents

AI Declaration.....	3
1 Summary	8
2 Poster	9
3 Introduction	10
4 Smart Parking and Object Detection Background	11
4.1 Smart Parking Systems.....	11
4.2 Object Detection	12
4.3 YOLO for Vehicle Detection	13
4.4 Consideration of Alternative Detection Models.....	14
4.5 Parking Occupancy Detection	14
4.6 Summary	15
5 Pathfinding and System Technologies Background.....	16
5.1 Route Planning and A* Pathfinding	16
5.2 FastAPI Backend.....	16
5.3 MongoDB Motor Storage.....	16
5.4 Frontend Technologies	17
5.5 Supporting Tools	17
5.6 Summary	17
6 Project Plan	18
7 Stages in the Parking Finder System	20
8 Project Architecture.....	21
9 Backend Implementation.....	22
9.1 Backend Role in the System.....	22

9.2	Upload Handling	22
9.3	Request Coordination	22
9.4	Integration with Detection and Pathfinding	23
9.5	Backend Upload and API Handling	24
10	Database and Data Storage	25
10.1	Why Storage Was Needed	25
10.2	Why MongoDB Motor Was Chosen	25
10.3	What Was Stored	25
10.4	Value of the Database Layer	26
11	Parking Space Detection with YOLOv8	27
11.1	Purpose of the Detection Stage	27
11.2	Dataset Choice	27
11.3	Training Difficulties	27
11.4	Detection Metrics and Vehicle Centre Extraction	28
11.5	Practical Detection Output	30
12	Parking Space Configuration and Occupancy Logic	30
12.1	Need for Parking Configuration	30
12.2	Polygon Based Space Definitions	30
12.3	Occupancy Classification	31
12.4	Importance to the Overall Project	33
13	Pathfinding and Route Generation	34
13.1	Why Pathfinding Was Included	34
13.2	Reuse of Previous Work	34
13.3	Grid Based Route Model	34

13.4	Limitations of the Grid Setup.....	35
13.5	Importance of the Pathfinding Stage.....	37
14	Frontend Interface and User Interaction.....	37
14.1	Purpose of the Frontend.....	37
14.2	Features of the Frontend.....	37
14.3	Raspberry Pi Touchscreen Use.....	37
14.4	Frontend Value	38
15	Testing and Evaluation.....	39
15.1	Testing Approach	39
15.2	Backend Testing.....	39
15.3	Detection Evaluation.....	39
15.4	Occupancy Testing.....	40
15.5	Pathfinding Testing.....	40
15.6	Integration Testing.....	41
15.7	What worked well.....	41
15.8	Limitations Observed During Testing.....	42
16	Problems and Challenges Faced	43
17	Ethics.....	44
18	Conclusion.....	45
19	References	47
20	Appendix.....	47

1 Summary

The goal of the Parking Finder project was to develop an intelligent parking system capable of analysing parking footage, detecting vehicle occupancy, and assisting users in locating available parking spaces. The purpose of the project was to apply machine learning and software engineering techniques to a practical real world problem that affects drivers in busy car parks.

The scope of this project focused on the development of a working parking analysis and navigation prototype. The main deliverables included vehicle detection, parking occupancy analysis, parking space configuration, backend processing, frontend interaction, Raspberry Pi image capture, and route generation to an available parking space.

The approach taken involved the use of a YOLOv8 based object detection model to identify vehicles in parking images [2], [3]. These detections were then compared against configurable parking regions so that spaces could be classified as occupied or available. Once free spaces had been identified, a grid based A* pathfinding system was used to calculate a route to a selected free parking bay [4].

The project was developed using FastAPI for the backend [5], MongoDB Motor for storage [6], Next.js and React for the frontend [7], [8], and OpenCV and PyTorch to support image processing and machine learning tasks [9], [10]. A C++ implementation of A* was integrated to handle pathfinding.

The main goal of developing a functional smart parking prototype was successfully achieved. The finished system can upload parking footage, detect vehicles, classify parking spaces, configure parking polygons, display the results through a frontend interface, and generate a route to an available space. The project demonstrates that machine learning, backend services, hardware integration, data storage, and pathfinding can be combined into one practical engineering solution.

2 Poster

INTRODUCTION

Finding parking at ATU is a common issue for students, especially during busy periods of the academic year when spaces are limited. This can lead to wasted time and added stress around campus car parks.

This project explores how algorithms and computer vision can be used in a smart parking system to detect available and occupied spaces, then guide drivers to the nearest free spot. It provides a practical and low cost alternative to traditional sensor based parking solutions.



Ollscoil
Teicneolaíochta
an Atlántaigh

Atlantic
Technological
University

THE SOLUTION

The proposed solution uses an overhead camera and a YOLOv8 model trained on aerial footage to analyse a car park image and determine which spaces are occupied. Predefined parking spot polygons are used to compare detections against known parking locations in real time.

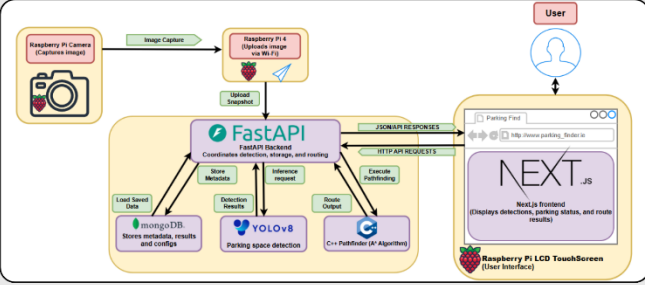
Once free spaces have been identified, a C++ A* pathfinding algorithm calculates the shortest route from the entrance to the nearest available spot. This result is then presented through a web interface using colour coded overlays and route guidance.

PARKING FINDER

Sean Maloney

BEng (Hons) Software & Electronic Engineering

ARCHITECTURE DIAGRAM



HOW IT WORKS

- Raspberry Pi Camera Module 3 captures images at regular intervals.
- The Raspberry Pi 4 uploads the image to the backend over Wi-Fi.
- YOLOv8 detects vehicles in the image.
- Each detected vehicle centre is checked against predefined parking spot polygons.
- Parking spaces are classified as occupied or available.
- Free spaces are mapped onto a 2D grid.
- A C++ A* algorithm calculates the shortest route to the nearest available space.
- The frontend displays the parking status and generated route.

TECHNOLOGIES

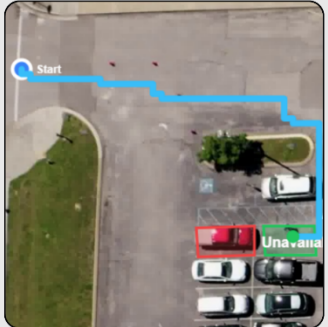
Front end: Next.js, React, Type Script, Tailwind CSS
Backend: FastAPI, Python, Uvicorn
AI/ML Vision: YOLOv8, Ultralytics, OpenCV, Pytorch
Database: MongoDB, Motor
Algorithms: A* Pathfinding, C++

CONCLUSION

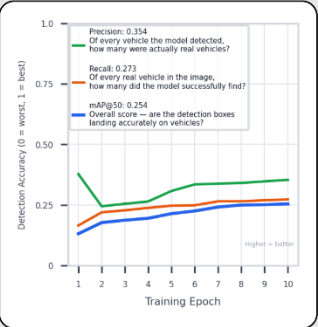
This project demonstrates that smart parking can be implemented using AI and software, without relying on costly sensor based infrastructure. It combines vehicle detection, parking space classification, and route generation into one working solution.

Future improvements could include moving from periodic image capture to live video streaming, along with real time campus deployment.

USER VIEW



YOLOv8 RESULTS



Training Epoch	Precision (0 = worst, 1 = best)	Recall (0 = worst, 1 = best)	mAP@50 (0 = worst, 1 = best)
1	0.354	0.273	0.254
2	0.354	0.273	0.254
3	0.354	0.273	0.254
4	0.354	0.273	0.254
5	0.354	0.273	0.254
6	0.354	0.273	0.254
7	0.354	0.273	0.254
8	0.354	0.273	0.254
9	0.354	0.273	0.254
10	0.354	0.273	0.254

3 Introduction

The aim of this project was to develop Parking Finder, a prototype system designed to analyse parking footage, detect vehicle occupancy, and help users locate available parking spaces. The motivation for the project came from the common difficulty of finding parking in busy car parks, where drivers can spend unnecessary time searching for a free space. This makes parking a practical problem that can benefit from intelligent automation and computer vision [1].

To address this problem, the project combines machine learning with software engineering. A YOLOv8 based object detection model is used to identify vehicles in uploaded video and captured image data [2], [3]. These detections are then compared against predefined parking space regions so that individual spaces can be marked as occupied or available. This allows the system to move beyond simple vehicle detection and provide useful parking information based on the actual layout of the parking area.

The project also includes a FastAPI backend, which is responsible for processing requests, handling uploads, and coordinating system operations [5]. MongoDB Motor is used for storing parking configurations, captures, and other project data [6]. In addition to occupancy detection, the system includes route generation to an available parking space, making the project a more complete parking assistance prototype rather than just a detection system.

The system also includes a hardware element through the use of a Raspberry Pi camera, Raspberry Pi 4, and LCD touchscreen. This helped make the project feel more like a complete product rather than just a software demonstration. Captured images could be uploaded from the Raspberry Pi to the backend over Wi-Fi, and the frontend interface could then display parking status and route results to the user.

The final result is a system that combines machine learning, backend services, data storage, frontend interaction, hardware integration, and route planning within one application. This report outlines the background to the project, the architecture of the system, the main stages of implementation, the challenges faced during development, and the conclusions drawn from the completed work.

4 Smart Parking and Object Detection Background

4.1 Smart Parking Systems

Smart parking systems are designed to reduce the time spent searching for parking spaces and to improve the efficiency of parking facilities. In busy environments such as university campuses, drivers often waste time looking for available spaces, which leads to delays, frustration, and unnecessary congestion. Smart parking research has shown that software driven systems can improve parking awareness and reduce inefficiency by identifying available spaces and presenting that information clearly to users [1].

Traditional parking systems often rely on dedicated sensors placed in each bay. While these systems can be effective, they can also be expensive to deploy and maintain. In contrast, a camera based system can monitor a wider area and use computer vision to detect multiple spaces at once. This makes a vision based approach more attractive for a student project and potentially more cost effective for real deployment.

In the context of this project, the aim was to create a practical smart parking prototype rather than a full commercial deployment. Even so, the same general goal applies, identify available spaces and help the user reach one quickly and clearly.

4.2 Object Detection

Object detection is a computer vision task that identifies the location and class of objects within an image or video stream. A detection model typically returns bounding boxes, class labels, and confidence values for the detected objects [2], [3]. In a parking system, this is useful because the system first needs to detect where the vehicles are before it can reason about parking occupancy.

Object detection differs from image classification because it does not just say what is in an image. It also identifies where those objects are located. This is essential for parking analysis because the position of a vehicle relative to a parking bay determines whether that bay is occupied.



Figure 2: Example of vehicle detection output in a parking scene

4.3 YOLO for Vehicle Detection

The detection approach used in this project is based on YOLOv8, from the Ultralytics YOLO family [3]. YOLO is one of the most widely used object detection approaches because it treats object detection as a direct prediction problem and is designed for speed as well as practicality [2].

This made YOLOv8 a good fit for Parking Finder. The system needed to process parking images and identify vehicles reliably enough for the later occupancy stage. Because the intended parking images were closer to an overhead or elevated viewpoint, the model also needed to work well on aerial style scenes rather than only street level images.

The project therefore used aerial style training data to align the model more closely with the intended use case. This became one of the major machine learning decisions in the development of the project.

4.4 Consideration of Alternative Detection Models

Before choosing YOLOv8, a number of alternative object detection models were considered during the research stage of the project. Since the system needed to detect vehicles from an elevated parking view and also fit into a wider smart parking pipeline, the model had to be practical as well as effective.

Approaches such as Faster R-CNN, SSD, and newer YOLO variants were considered. Faster R-CNN is known for strong detection performance, but it appeared heavier and less practical for this project. SSD was also reviewed, but it did not seem to offer the same balance between performance and usability as YOLOv8.

YOLOv11 was also looked at during this stage. However, at the time of development, it did not appear to be as well documented or as widely supported as YOLOv8. Since this project required a model that could be trained, tested, and integrated without adding unnecessary risk, documentation and available guidance were important factors.

Overall, YOLOv8 was chosen because it offered the best balance between detection capability, practicality, and ease of integration for the project.

4.5 Parking Occupancy Detection

Parking occupancy detection goes beyond general vehicle detection by deciding whether a specific parking space is occupied or available. This requires the system to know not only where vehicles are, but also where each parking bay is located in the image.

For this reason, parking occupancy systems often rely on predefined slot boundaries or parking regions [1]. In this project, configurable parking polygons were used to represent each parking bay. Vehicle detections were then compared against these polygons to determine whether a space should be marked as occupied or free.

This approach was important because it made the system adaptable. Rather than assuming one fixed parking layout, the system could be configured to suit a particular camera view and car park arrangement.

4.6 Summary

The background research for this project shows that smart parking is a strong application area for computer vision and software engineering. Object detection provides the ability to identify vehicles, while occupancy logic turns those detections into useful parking information. These ideas provide the foundation of the Parking Finder system.

5 Pathfinding and System Technologies Background

5.1 Route Planning and A* Pathfinding

Route planning is an important part of this project because the system was designed not only to detect available parking spaces, but also to guide the user towards one. To do this, a pathfinding algorithm is needed to calculate a route between two points.

The algorithm used in this project was A*, which is one of the most established algorithms for shortest path style search problems [4]. A* works by combining the known cost of the path travelled so far with a heuristic estimate of the remaining cost to the goal. This allows it to find efficient routes while avoiding unnecessary search in many cases.

For Parking Finder, the use of A* meant that once a free parking bay had been identified, a route could be generated from the entrance or starting point to that space. This significantly improved the usefulness of the system compared to a detection only prototype.

5.2 FastAPI Backend

The backend of the project was built using FastAPI [5]. FastAPI is a Python framework used for building APIs and backend services. It was suitable for this project because it allowed the system to be organised around clearly defined API endpoints and backend processes.

FastAPI handled image uploads, processing requests, parking configuration data, and communication with both the detection and pathfinding parts of the system. This made it the central coordination point of the project.

5.3 MongoDB Motor Storage

The project uses MongoDB Motor as its database layer [6]. MongoDB Motor stores data in flexible BSON documents, which makes it well suited to applications where configuration and metadata can vary in structure.

In Parking Finder, MongoDB Motor was used to store parking configurations, image metadata, capture related records, and other supporting data. This allowed the system to preserve configurations and reuse them rather than treating each session as completely temporary.

5.4 Frontend Technologies

The frontend was developed using Next.js and React [7], [8]. These technologies were chosen because they support modern component based interface design and are well suited to building interactive web applications.

In this project, the frontend had to support several different interaction types. These included viewing results, configuring parking polygons, uploading data, and displaying route output. A clear frontend was important because the project was intended to be demonstrated visually rather than existing only as backend code.

5.5 Supporting Tools

The wider project also made use of OpenCV [9] and PyTorch [10]. OpenCV supported image related processing tasks, while PyTorch supported machine learning related workflows around the YOLO model and training environment.

5.6 Summary

The technologies selected for this project were chosen to support a complete end to end system. FastAPI handled backend coordination, MongoDB Motor supported storage, Next.js and React provided the user interface, YOLOv8 supported vehicle detection, and A* provided route generation.

6 Project Plan

At the beginning of the project, the work was divided into clear stages so that progress could be managed more effectively. Since the project included machine learning, backend development, frontend development, database integration, Raspberry Pi hardware, and route generation, it was important to break the work into manageable tasks.

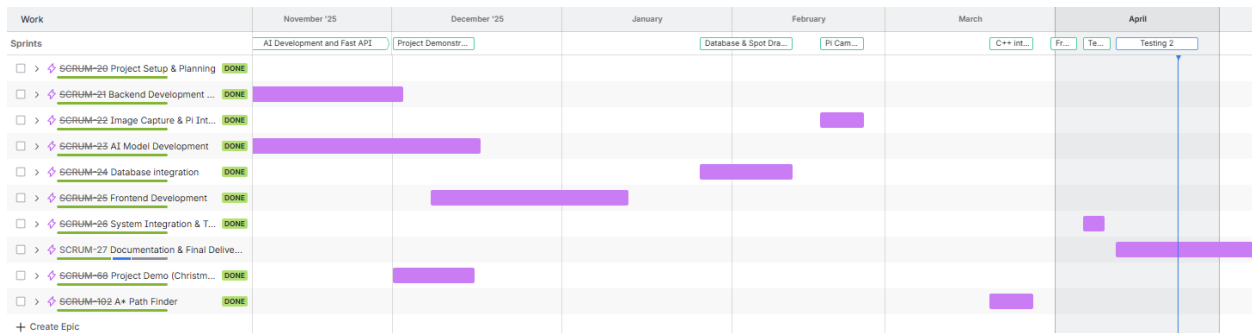


Figure 4: Project timeline showing the main development stages

The project was managed using Jira, and weekly stand ups were carried out with the class. This helped to track progress and maintain accountability throughout the year. It also made it easier to see which tasks had been completed, which were in progress, and which still needed attention.

The project originally began with a different idea. My initial plan was to use a drone to capture aerial footage of the parking area. However, after discussing the idea with my supervisor, that concept was changed. A more realistic and practical solution was to use a fixed or pre existing overhead camera setup instead. This turned out to be a better decision, as it made the project easier to justify, more practical to implement, and more realistic in terms of real world deployment.

As development continued, some parts of the project took longer than expected. One of the main examples was the C++ A* pathfinding system. This work had originally been part of another module, and adapting it so that it worked correctly inside this final year project took more time than expected. There were also additional features that had been planned, such as

login support, Docker containerisation, and Kubernetes based deployment ideas, but these were not completed due to time constraints.

Even though the plan changed during the year, the core system remained focused on the main smart parking functionality. This helped keep the project achievable while still allowing it to include a broad set of technologies and features.

7 Stages in the Parking Finder System

The overall method used in Parking Finder follows a staged process from image capture through to final route display. The system can be understood as a sequence of connected stages.

1) Image capture

A Raspberry Pi camera captures an image of the parking area.

2) Image upload

The Raspberry Pi 4 uploads the captured image to the backend over Wi-Fi.

3) Vehicle detection

The FastAPI backend passes the image through the YOLOv8 detection pipeline so that vehicles can be identified.

4) Parking space classification

The detected vehicle positions are compared against predefined parking polygons. Each space is then classified as occupied or available.

5) Free space selection

Once available spaces are known, one of these spaces is selected as the target destination.

6) Route generation

A grid based C++ A* implementation calculates a path from the chosen start point to the selected available parking space.

7) Frontend display

The frontend displays the occupancy result and the generated route to the user.

8 Project Architecture

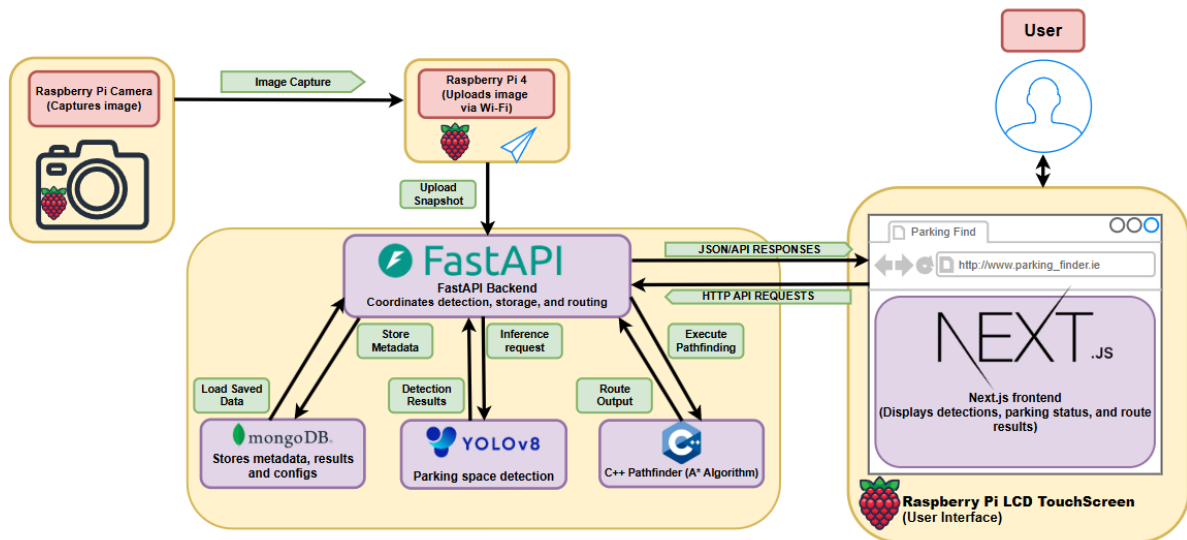


Figure 8-1 Architecture Diagram of the Parking Finder

The architecture of Parking Finder is made up of several connected components that support parking analysis and route generation. These include a Raspberry Pi camera, Raspberry Pi 4, FastAPI backend, MongoDB Motor, YOLOv8 detection pipeline, C++ A* pathfinding module, and a Next.js frontend.

The system begins with image capture. A Raspberry Pi camera captures the parking image, and the Raspberry Pi 4 uploads it over Wi-Fi. The FastAPI backend then acts as the central controller, handling image processing, metadata storage, saved configurations, and pathfinding requests.

The YOLOv8 component detects vehicles in the images, and these detections are compared against stored parking polygons to determine space occupancy. MongoDB Motor stores parking configurations, metadata, and related project data so that the system can retain important information between sessions.

Once free spaces have been identified, the backend calls the C++ A* module to generate a route. This is then returned to the Next.js frontend, shown on the Raspberry Pi LCD touchscreen, where the user can view parking status and route results.

Overall, the architecture was designed to separate the project into logical parts while keeping them integrated into one coherent system.

9 Backend Implementation

9.1 Backend Role in the System

The backend of Parking Finder was implemented using FastAPI [5]. It was responsible for coordinating the main operations of the project and allowing the other components to communicate properly.

The backend acted as the middle layer between hardware capture, AI detection, database storage, and the frontend. This role was critical because the project did not consist of a single program running in isolation. Instead, it was a connected system with multiple parts that needed to exchange information.

9.2 Upload Handling

One of the most important backend tasks was accepting image or media uploads. The backend needed to receive images coming from the Raspberry Pi or from the interface and then prepare them for processing.

This stage was important because it acted as the first step in the wider data pipeline. Once the image had reached the backend, the system could then trigger detection and later occupancy analysis.

9.3 Request Coordination

The backend also coordinated requests from the frontend. This included returning stored parking information, receiving requests for route generation, and sending processed results back to the interface.

Using FastAPI for this made sense because the API based design kept the frontend and backend clearly separated. The frontend was responsible for display and interaction, while the backend handled the heavier logic.

9.4 Integration with Detection and Pathfinding

The backend also played a central role in linking the detection system and the pathfinding system. It passed image data to the detection stage and triggered route generation once a destination space had been identified.

Without this backend layer, the project would have been much harder to organise. FastAPI helped make the system modular while still keeping it connected.

9.5 Backend Upload and API Handling

A key part of the backend was handling image uploads from the Raspberry Pi. This allowed the system to receive the latest parking image, save it on the server, and store a reference to it for later use. This was important because the parking analysis pipeline depended on current image data being available before any detection or route generation could take place.

The backend was implemented using FastAPI, and one of the clearest examples of this can be seen in the image upload route. This route accepts an uploaded file, checks that the file type is valid, saves it to the capture directory, and updates the database with the latest snapshot information. This shows how the backend acts as the communication layer between the Raspberry Pi capture stage and the rest of the system.

```
@router.post("/pi/upload-snapshot")
async def upload_snapshot_from_pi(
    file: UploadFile = File(...),
    camera_id: str = Form("cam1"),
    db: AsyncIOMotorDatabase = Depends(get_db),
):
    if not file.filename:
        raise HTTPException(status_code=400, detail="no file uploaded")

    ext = Path(file.filename).suffix.lower() or ".jpg"
    if ext not in [".jpg", ".jpeg", ".png"]:
        raise HTTPException(status_code=400, detail="file must be jpg, jpeg, or png")

    save_path = CAPTURE_DIR / f"{camera_id}_latest{ext}"
    file_bytes = await file.read()
    await asyncio.get_event_loop().run_in_executor(None, save_path.write_bytes, file_bytes)

    await db.captures.update_one(
        {"_id": camera_id},
        {"$set": {"_id": camera_id, "camera_id": camera_id, "file": save_path.name, "updated_at": datetime.utcnow()}},
        upsert=True,
    )
    return {"status": "ok", "camera_id": camera_id, "image_url": f"/api/cameras/{camera_id}/image"}
```

Code Snippet 9-1: FastAPI Route used to upload the latest snapshot from the Raspberry Pi

This snippet demonstrates several useful backend responsibilities in one place. First, it validates that a supported image file has been uploaded. It then saves the latest image using the camera identifier, updates the MongoDB Motor captures collection with the latest file information, and returns a JSON response containing the image URL. This made it possible for the rest of the system to access the most recent parking snapshot in a structured way.

10 Database and Data Storage

10.1 Why Storage Was Needed

A database was needed because the project required more than temporary runtime data.

Parking polygons, image metadata, capture information, and route related details all needed to be stored so that they could be reused later.

If the project had relied only on in memory or temporary data, important configuration information would have been lost between uses of the system.

10.2 Why MongoDB Motor Was Chosen

MongoDB Motor was chosen because it works well with flexible document style data [6].

Parking related data does not always fit neatly into rigid table structures, especially in a prototype system where configurations can change during development.

Using MongoDB Motor made it easier to store polygon definitions, metadata, capture information, and other project records in a way that could evolve during the development process.

10.3 What Was Stored

The database was used to store:

- parking configuration data
- capture or upload related data
- metadata about processing
- saved information required by the frontend and backend

This gave the system continuity and meant the same parking layout could be loaded again instead of recreated each time.

10.4 Value of the Database Layer

The database layer added a lot of value to the project because it made the whole system feel more complete and structured. Rather than behaving like a temporary one off script, Parking Finder functioned more like a proper application with reusable data and persistent configuration. This was important because the system needed to retain information such as parking layouts, camera related settings, and stored metadata between sessions instead of requiring everything to be recreated each time it was used.

Having a database also improved the practicality of the project. It meant that parking configurations could be saved once and loaded again when needed, which made the system more efficient and easier to manage during testing and development. Overall, the database layer helped support continuity, organisation, and the long term usability of the system.

```
_id: ObjectId('698503be14b19279eac2d6b9')
camera_id: "cam1"
▼ frame_size: Object
  w: 1280
  h: 720
▼ spots: Array (7)
  ▶ 0: Object
  ▶ 1: Object
  ▶ 2: Object
  ▶ 3: Object
  ▶ 4: Object
  ▶ 5: Object
  ▶ 6: Object
version: 1
status: "draft"
created_at: 2026-02-05T20:55:26.443+00:00
```

Figure 10-1: Example of stored parking configuration data

11 Parking Space Detection with YOLOv8

11.1 Purpose of the Detection Stage

The vehicle detection stage was one of the most important technical parts of the project. Before the system could decide whether a parking space was free or occupied, it first needed to detect the vehicles present in the parking image.

This stage was built around YOLOv8, which was used because it is a practical object detection model for real world applications [2], [3].

11.2 Dataset Choice

The model was trained using an Ultralytics aerial drone style dataset, because the project was aimed at an overhead or elevated parking view. This matched the real intended use of the project more closely than a standard street view vehicle dataset would have done.

Even though the project later moved away from using a drone directly, the aerial style viewpoint still remained relevant because the chosen camera perspective was elevated.

11.3 Training Difficulties

One of the main difficulties during model development came from the training data. Since the system was intended to detect vehicles from an elevated parking view, the quality and suitability of the dataset had a direct effect on the final detection performance. Early attempts to reduce the dataset so that the model would focus more narrowly on vehicles did not improve the results as expected. Instead, reducing the dataset removed useful variation from the training data and weakened the overall training process.

This showed that dataset quality was not simply about having fewer, more targeted images, but about having enough suitable examples and enough variety for the model to generalise properly. As a result, the full dataset was used for the final training process. Even then, the final metrics remained fairly modest, which suggests that stronger or more specialised parking focused training data would likely be needed to improve detection performance further.

11.4 Detection Metrics and Vehicle Centre Extraction

In order to use the YOLOv8 detections for parking analysis, the system first needed to determine the position of each detected vehicle within the image. A detection bounding box can be represented using two corner points, (x_1, y_1) and (x_2, y_2) . From this, the centre point of the detection can be calculated using:

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2}$$

This was important in the project because the centre point of each detected vehicle was later checked against predefined parking polygons in order to decide whether a parking space was occupied.

To evaluate the performance of the detection model, standard object detection metrics were considered. Two of the most useful are **precision** and **recall**:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

These metrics helped to describe how well the trained model was identifying vehicles in the parking images. Although the final results were not as strong as originally hoped, they still provided a useful indication of model performance and highlighted the effect that dataset quality had on training.

The code used in the project reflects this process clearly. The model is loaded, inference is run on the input image, and vehicle points are extracted from the returned detections. This output then becomes the input to the parking occupancy stage.

```

result = model.predict(image, conf=conf, verbose=False, device=YOLO_DEVICE)[0]

points: List[Tuple[int, int]] = []
if result.bboxes is not None and len(result.bboxes) > 0:
    xyxy = result.bboxes.xyxy.cpu().numpy()
    cls = result.bboxes.cls.cpu().numpy().astype(int) if result.bboxes.cls is not None else None
    for i, (x1, y1, x2, y2) in enumerate(xyxy):
        if vehicle_class_ids is not None and cls is not None and cls[i] not in vehicle_class_ids:
            continue
        points.append(_bottom_center(x1, y1, x2, y2))

```

Code Snippet 11-1: YOLOv8 inference and extraction of vehicle detection points

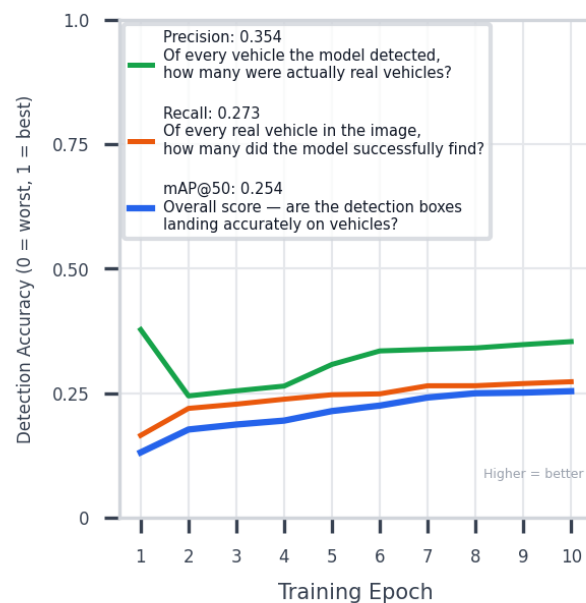


Figure 11-2: YOLO training results showing precision, recall and mAP across 10 epochs

The graph shows that the model improved somewhat over time, although the overall performance remained fairly modest. Earlier attempts to reduce the dataset did not lead to better results, so the full dataset was used in the final training process. This gave a more stable basis for training, but the final metrics still showed that the model had clear room for improvement.

11.5 Practical Detection Output

While the training metrics showed that the model performance remained fairly modest, the detection stage still produced useful results within the parking scenes used for the prototype. In practical terms, the model was able to identify vehicles well enough to support the later occupancy classification stage of the system.

This was important because the goal of the project was not only to train a detection model in isolation, but to integrate that model into a wider smart parking pipeline. Even where detections were not perfect, the output was still sufficient to allow the system to mark spaces as occupied or available and continue to the pathfinding stage.

The practical detection output therefore demonstrated that the model was usable within the prototype, even though there was still clear room for improvement in accuracy and consistency.



Figure 11-3: Example vehicle detection result produced by YOLOv8

12 Parking Space Configuration and Occupancy Logic

12.1 Need for Parking Configuration

Vehicle detection alone cannot identify whether a specific parking bay is occupied. The system must also know where the parking spaces are located.

This is why Parking Finder uses configurable parking polygons. Each parking bay is manually represented as a region in the image.

12.2 Polygon Based Space Definitions

Each polygon represents the boundary of an individual parking space. Once these have been defined, the system can compare vehicle detections against them and determine occupancy.

This approach made the system much more flexible than a hard coded layout would have been. It meant the parking layout could be adjusted to suit a specific image or camera view.

12.3 Occupancy Classification

After vehicles had been detected, the next stage of the project was to decide whether a specific parking space was occupied or available. This was done by comparing each detected vehicle point against a predefined parking polygon.

The occupancy logic can be described as:

$$Occupied(s) = \begin{cases} 1, & \text{if there exists a detected point } p \text{ such that } p \in P_s \\ 0, & \text{otherwise} \end{cases}$$

Where P_s represents the polygon region for parking space(s)

This was a practical choice for the prototype because it allowed the project to convert raw object detections into actual parking availability information without requiring a more complicated overlap calculation. Once the centre point of a detected vehicle was found, the system only needed to decide whether that point lay inside the region of a parking bay.

This logic is directly visible in the project code. The polygon is first converted from normalised coordinates into image space coordinates, then a point in polygon test is used to check whether any detected vehicle point lies inside that parking region. If so, the space is marked as occupied. Otherwise, it is marked as available.

```

def _norm_poly_to_px(poly: List[Dict[str, float]], w: int, h: int) -> np.ndarray:
    pts = [(int(p["x"] * w), int(p["y"] * h)) for p in poly]
    return np.array(pts, dtype=np.int32).reshape((-1, 1, 2))

def _is_point_in_poly(px: int, py: int, poly_np: np.ndarray) -> bool:
    return cv2.pointPolygonTest(poly_np, (float(px), float(py)), False) >= 0

def _bottom_center(x1: float, y1: float, x2: float, y2: float) -> Tuple[int, int]:
    return int((x1 + x2) / 2.0), int((y1 + y2) / 2.0)

for s in spots_doc.get("spots", []):
    sid = s.get("id", "")
    poly = s.get("polygon", [])
    if not sid or len(poly) < 3:
        continue

    poly_np = _norm_poly_to_px(poly, w, h)
    is_occupied = any(_is_point_in_poly(px, py, poly_np) for px, py in points)

    spots.append({
        "id": sid,
        "type": s.get("type", "parking"),
        "description": s.get("description", ""),
        "status": "occupied" if is_occupied else "available",
    })

```

Code Snippet 12-1: Classifying parking occupancy by checking detected vehicle points against parking polygons

This snippet shows the main occupancy logic used in the project. First, each parking polygon is converted from normalised coordinates into image space coordinates. A point in polygon test is then used to check whether any detected vehicle point lies inside that parking region. If a point is found inside the polygon, the space is marked as occupied; otherwise, it is marked as available.

12.4 Importance to the Overall Project

This section of the project is where the detection system becomes a true parking system.

Without occupancy logic, the project would only be able to say that vehicles exist in the image.

With occupancy logic, it can actually say which spaces are free and which are occupied.



Figure 12-1: Parking space configuration using predefined polygon regions



Figure 12-2: Parking occupancy result after comparing detections to configured parking spaces

13 Pathfinding and Route Generation

13.1 Why Pathfinding Was Included

One of the features that made the project more interesting was the addition of route generation. Instead of simply telling the user that a space was free, the system could also guide them towards that space.

This made the system feel more complete and more useful from a user point of view.

13.2 Reuse of Previous Work

This was also the part of the project I was most proud of. The C++ A* implementation had originally been created as part of another module, and I was able to adapt and integrate it into my final year project. Being able to connect work from another module into this project made it especially rewarding once it worked.

13.3 Grid Based Route Model

The route generation system used a grid based representation of the parking environment. The entrance or start point and the selected parking destination could then be represented within that grid.

From there, the A* algorithm could calculate a path from start to destination.

13.4 Limitations of the Grid Setup

One of the main limitations of the pathfinding stage was that the grid had to be configured very carefully in order for the generated route to be valid. The A* algorithm itself worked correctly once the grid was set up, but the quality of the route depended heavily on how accurately the parking environment was translated into the grid representation.

In practice, this meant that the entrance position, parking bay locations, driveable areas, and blocked cells all needed to be mapped consistently. If the grid layout did not match the parking image closely enough, the algorithm could produce a route that was unrealistic, inefficient, or in some cases fail to find a valid path at all. Small setup errors, such as incorrectly marking obstacles or misaligning the intended destination cell, could therefore affect the final route output.

This limitation is important because it shows that pathfinding performance in the system was not only dependent on the A* algorithm itself, but also on the quality of the manual grid configuration used as input. If I had more time, I would improve this part of the project by making the grid setup process more automated, more accurate, and less dependent on manual adjustment.

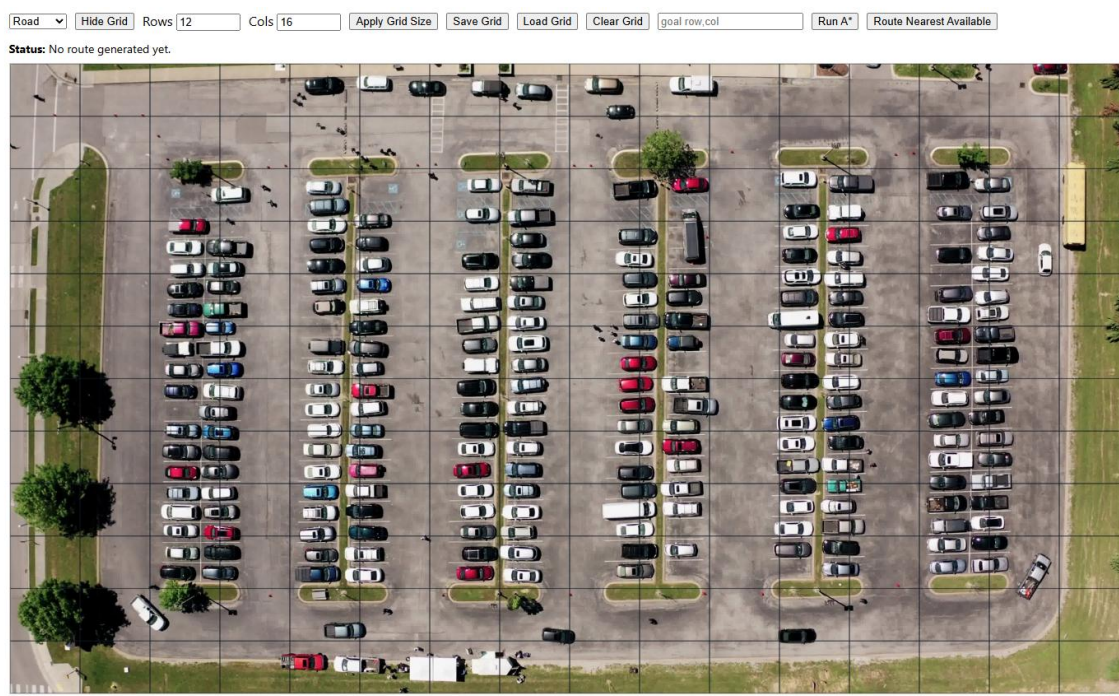


Figure 13-1: Grid representation used for route generation



Figure 13-2: Route generated from the entrance to an available parking space

13.5 Importance of the Pathfinding Stage

The route planning stage added technical depth to the project. It showed that Parking Finder was not just an AI demo, but a wider engineering system combining computer vision, configuration logic, backend services, and algorithmic path generation.

14 Frontend Interface and User Interaction

14.1 Purpose of the Frontend

The frontend of Parking Finder was built using Next.js and React [7], [8]. Its purpose was to provide an accessible user interface for interacting with the system.

The frontend allowed the user to view parking status, interact with parking setup, and see route generation output in a visual way.

14.2 Features of the Frontend

The frontend supported:

- parking media upload
- viewing parking status
- configuration of parking polygons
- display of route results
- user interaction through a touchscreen interface

This made the project much easier to demonstrate and much more understandable than if the results had only been shown through backend output.

14.3 Raspberry Pi Touchscreen Use

The frontend was displayed using a Raspberry Pi LCD touchscreen, which helped the project feel more like a complete hardware software prototype.

This also tied the project together visually, because the same system that captured parking information could also present the processed results in a clear and interactive way.

14.4 Frontend Value

The frontend was important because it turned the technical backend and AI processing into something the user could actually interact with. It made the project feel like one coherent application rather than several disconnected technical parts.

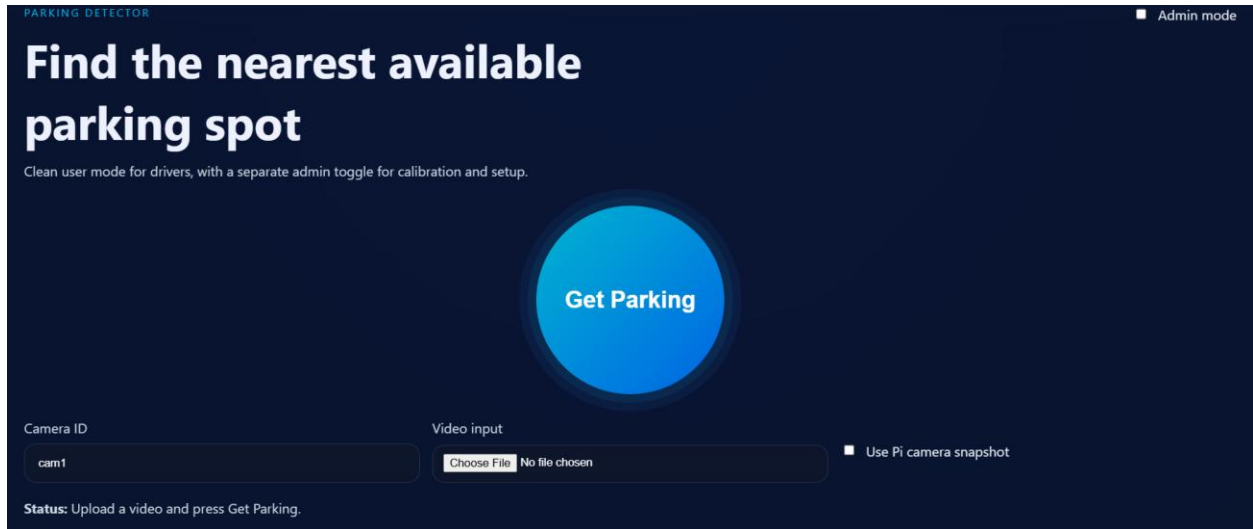


Figure 14-1: Main frontend interface of Parking Finder

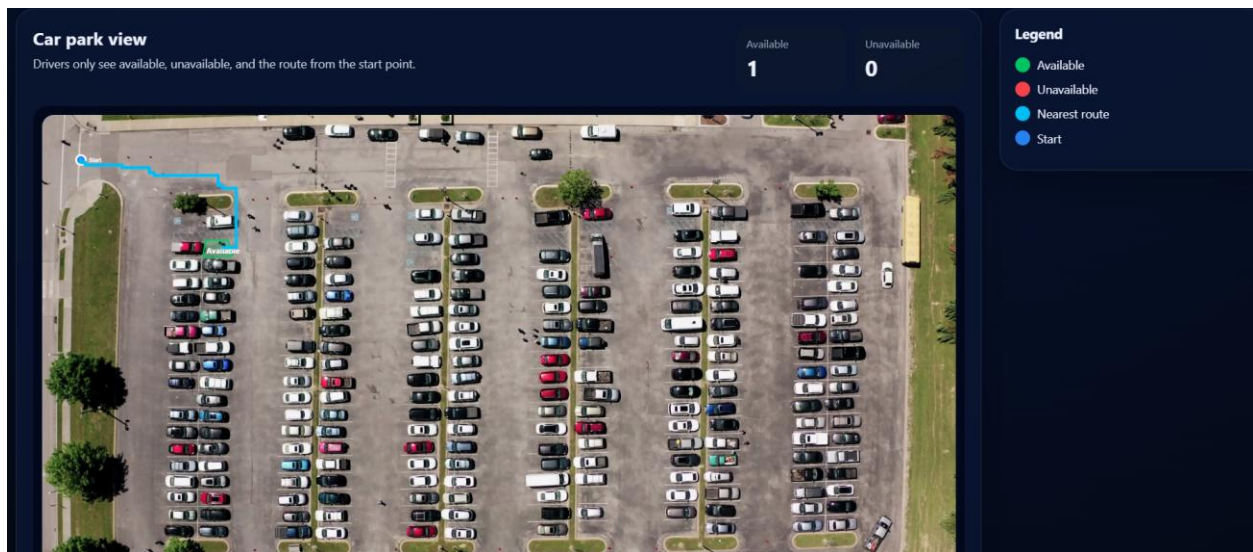


Figure 14-2: Frontend display of parking status and route output

15 Testing and Evaluation

15.1 Testing Approach

Testing in this project involved a mixture of practical system testing, backend validation, and output evaluation. Since the project included a number of interacting parts, it was not enough to test only the machine learning model in isolation.

Different areas of the system had to be checked separately and then together as an integrated prototype.

15.2 Backend Testing

The backend was tested to ensure that uploads, stored data retrieval, parking configurations, and processing requests could be handled correctly. API behaviour was checked during development, and pytest style backend testing was considered as part of the development process.

The main goal of backend testing was to confirm that the service layer could correctly accept requests and return meaningful results without breaking the overall system flow.

15.3 Detection Evaluation

The YOLO training graph provided a direct evaluation of the detection model. Precision, recall, and mAP@50 were used as the main indicators of performance.

Although the resulting values were not especially high, they still offered useful insight. They showed that the model could detect vehicles to a usable degree, but also that the dataset and training process placed clear limits on final performance.

The detection evaluation discussed here is based on the training results already shown in Figure 11.2.

15.4 Occupancy Testing

The occupancy stage was tested by comparing detections with configured parking polygons and observing whether spaces were marked correctly as occupied or available.

This worked best when the parking polygons were carefully configured and the detections were accurate. It also highlighted how dependent the occupancy stage was on both good configuration and sufficiently good detection.

15.5 Pathfinding Testing

The pathfinding stage was tested by selecting a route start point and checking whether the A* system could produce a valid path to the chosen available parking space. In the final prototype, the algorithm was able to generate routes successfully, but the results also showed that the quality of the output depended heavily on the grid setup used to represent the parking area.

The main issue was not with the A* search itself, but with how the parking environment was mapped into the grid. For pathfinding to work correctly, driveable areas, blocked cells, the entrance point, and destination spaces all had to be placed accurately. If the grid did not match the real layout closely enough, the generated route could become unrealistic or the system could fail to return the expected path. This showed that route generation was strongly dependent on careful configuration of the underlying grid model.

Overall, the testing showed that the pathfinding system was functional, but also highlighted that the reliability of route generation was closely tied to the quality of the grid representation.

15.6 Integration Testing

One of the most important parts of testing was integration testing. This involved checking that:

- the Raspberry Pi capture stage worked
- uploads reached the backend
- detection results were returned correctly
- parking occupancy was displayed correctly
- route generation could be triggered
- the frontend showed the results properly

This integration testing was critical because the value of the project came from the full system working together, not just from one isolated component.

15.7 What worked well

The main parts that worked well in the final system were:

- parking media upload
- vehicle detection
- parking space classification
- parking polygon configuration
- route generation
- frontend interaction
- Raspberry Pi capture workflow

This meant that the main project objective was achieved

15.8 Limitations Observed During Testing

The biggest limitations observed during testing were:

- modest model training performance
- sensitivity of detection quality to the training dataset
- dependence of route generation on accurate grid setup
- reliance on careful manual configuration of parking regions and routing inputs

In particular, the testing showed that model performance was influenced by the quality and suitability of the training data. While the final detector was usable within the prototype, the results remained fairly modest, showing that stronger or more specialised training data would likely improve accuracy.

The testing also showed that the routing stage depended heavily on how well the real parking layout was represented in the grid. Even though A* itself functioned correctly, inaccurate or inconsistent grid setup could reduce the quality of the generated route. Even with these limitations, the system still functioned successfully as a prototype and demonstrated the overall concept clearly.

16 Problems and Challenges Faced

One of the hardest technical issues was working through CUDA, PyTorch, and compatibility related setup problems on the machine learning side. Researching the correct versions of CUDA, drivers, and related dependencies took a significant amount of time and was one of the more frustrating parts of the project, especially because it was necessary before meaningful training work could even begin.

Another major challenge came from the dataset and training process. At one point, I attempted to reduce the dataset so that the model would focus more specifically on vehicles. However, because many of the original images contained vehicles alongside other objects, the dataset became too reduced and less effective overall. This meant that even after increasing the number of epochs, the model no longer had enough strong training data to improve properly.

The A* pathfinding work also took more time than expected. Although I was very happy with the final result, integrating a C++ pathfinding implementation from another module into this project required extra effort and debugging.

The project also changed direction from its original idea. At the beginning, the plan was to use a drone for aerial footage. However, this was changed after discussion with the supervisor, and the project moved towards using a more realistic camera based approach. In hindsight, this was the right decision, as it made the project much more practical and believable.

Finally, there were features that I wanted to include but did not complete due to time. These included login functionality, containerisation, Docker support, and Kubernetes related deployment ideas. While these would have added more software deployment depth, I prioritised the core smart parking functionality first.

17 Ethics

Although the project was mainly technical, there are still ethical and practical considerations associated with it.

A camera based parking system may capture vehicles and, depending on the viewpoint, may also capture people. This creates privacy considerations around image storage, data handling, and system usage. Any real world deployment of such a system would need clear rules around what data is stored and for how long.

There is also the issue of system reliability. If a parking space is incorrectly marked as free when it is actually occupied, that could inconvenience the user. Similarly, if route guidance is based on an incorrect grid or inaccurate occupancy result, it may become less useful.

These considerations mean that while the prototype demonstrates the concept successfully, any real deployment would need stronger attention to privacy, validation, and reliability.

18 Conclusion

The aim of this project was to develop a smart parking prototype capable of detecting vehicles, identifying free and occupied spaces, and guiding the user to an available parking space. This goal was successfully achieved through the development of Parking Finder.

The final system combines a Raspberry Pi image capture setup, FastAPI backend, MongoDB Motor storage, YOLOv8 vehicle detection, configurable parking polygons, C++ A* pathfinding, and a Next.js frontend. Together, these components form a working end to end prototype that demonstrates how machine learning and software engineering can be combined to solve a practical parking problem.

One of the strongest parts of the project was the successful integration of multiple technologies into one connected system. In particular, the A* route generation component allowed the project to go beyond simple vehicle detection and provide a more complete parking assistance function.

At the same time, the project also highlighted some important limitations. Detection performance was strongly influenced by the quality and suitability of the training dataset, while the effectiveness of route generation depended on how accurately the real parking layout was represented in the grid model used by the A* algorithm. These limitations do not take away from the overall success of the project, but they do show where further improvements would be needed in a more advanced version of the system.

Overall, Parking Finder demonstrates that a low cost smart parking system can be built using AI, backend services, route generation, preexisting infrastructures without relying entirely on expensive sensor based infrastructure. With more time, the project could be improved further through stronger model training, better grid configuration, live video support, and wider system deployment.

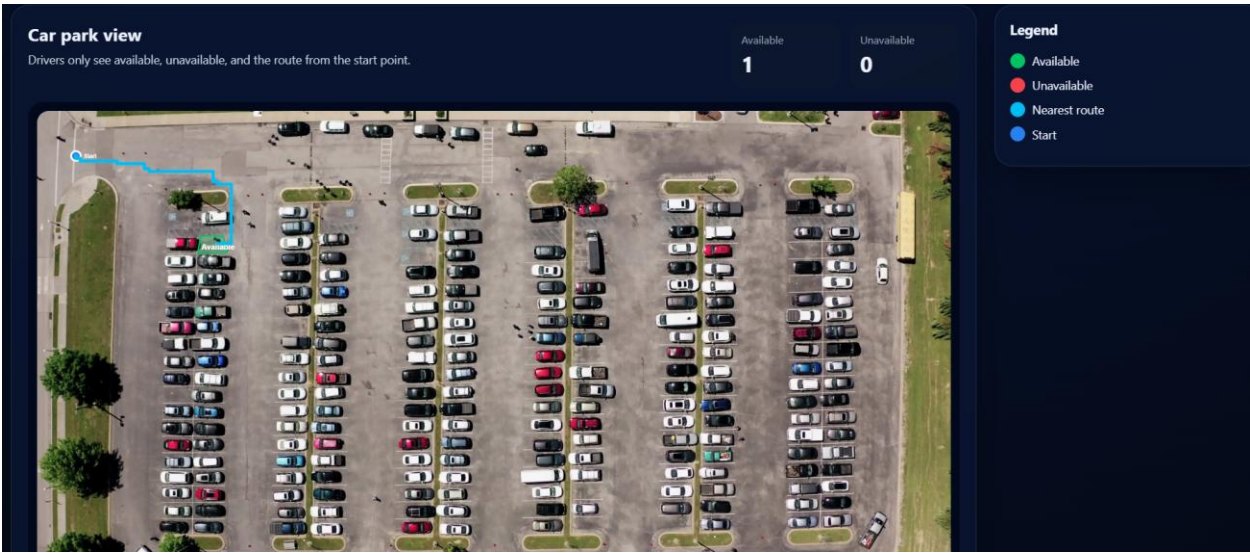


Figure 18-1: Final example of parking detection and route output

19 References

- [1] Ultralytics, “Ultralytics Solutions.” [Online]. Available: <https://docs.ultralytics.com/solutions/>
- [2] Ultralytics, “Ultralytics YOLO Docs.” [Online]. Available: <https://docs.ultralytics.com/>
- [3] Ultralytics, “Object Detection - Ultralytics YOLO Docs.” [Online]. Available: <https://docs.ultralytics.com/tasks/detect/>
- [4] NetworkX Developers, “astar_path — NetworkX documentation.” [Online]. Available: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.shortest_paths.astar.astar_path.html
- [5] FastAPI, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [6] MongoDB, Inc., “Tutorial: Using Motor With asyncio - Motor 3.7.1 documentation.” [Online]. Available: <https://motor.readthedocs.io/en/stable/tutorial-asyncio.html>
- [7] Vercel, “Next.js Documentation.” [Online]. Available: <https://nextjs.org/docs>
- [8] Meta, “React Documentation.” [Online]. Available: <https://react.dev/>
- [9] OpenCV, “OpenCV Documentation.” [Online]. Available: <https://docs.opencv.org/4.x/>
- [10] PyTorch, “PyTorch Documentation.” [Online]. Available: <https://docs.pytorch.org/docs/stable/index.html>

20 Appendix

Ultralytics 8.4.11 Python 3.12.10 torch 2.11.0.dev20260203+cu128 CUDA:0 (NVIDIA GeForce RTX 5070 Laptop GPU, 8151MiB)

engine\trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=8, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=datasets\VisDrone\data.yaml, degrees=0.0, deterministic=True, device=None, dfl=1.5, dnn=False, dropout=0.0,

dynamic=False, embed=None, end2end=None, epochs=10, erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8n.pt, momentum=0.937, mosaic=1.0, multi_scale=0.0, name=visdrone_parking_detector4, nbs=64, nms=False, opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False, project=None, rect=False, resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False, save_crop=False, save_dir=C:\Users\these\Desktop\FYP2025 SeanMaloney\runs\detect\visdrone_parking_detector4, save_frames=False, save_json=False, save_period= 1, save_txt=False, scale=0.5, seed=0, shear=0.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=val, stream_buffer=False, task=detect, time=None, tracker=botsort.yaml, translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None

Overriding model.yaml nc=80 with nc=10

	from	n	params	module	arguments
0	1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	1	1	7360	ultralytics.nn.modules.block.C2f	[32, 32, 1, True]
3	1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
4	1	2	49664	ultralytics.nn.modules.block.C2f	[64, 64, 2, True]
5	1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
6	1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]


```

7          1 1  295424 ultralytics.nn.modules.conv.Conv      [128, 256, 3, 2]
8          1 1  460288 ultralytics.nn.modules.block.C2f      [256, 256, 1, True]
9          1 1  164608 ultralytics.nn.modules.block.SPPF      [256, 256, 5]
10         1 1    0 torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
11        [ 1, 6] 1    0 ultralytics.nn.modules.conv.Concat    [1]
12         1 1  148224 ultralytics.nn.modules.block.C2f      [384, 128, 1]
13         1 1    0 torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
14        [ 1, 4] 1    0 ultralytics.nn.modules.conv.Concat    [1]
15         1 1   37248 ultralytics.nn.modules.block.C2f      [192, 64, 1]
16         1 1   36992 ultralytics.nn.modules.conv.Conv      [64, 64, 3, 2]
17        [ 1, 12] 1    0 ultralytics.nn.modules.conv.Concat    [1]
18         1 1  123648 ultralytics.nn.modules.block.C2f      [192, 128, 1]
19         1 1  147712 ultralytics.nn.modules.conv.Conv      [128, 128, 3, 2]

```

...

```

bus      131    251   0.486   0.323   0.341   0.238

```

```

motor    485    4886   0.342   0.323   0.253   0.0925

```

Speed: 0.1ms preprocess, 1.2ms inference, 0.0ms loss, 2.5ms postprocess per image

Results saved to <C:\Users\these\Desktop\FYP2025>

SeanMaloney\runs\detect\visdrone_parking_detector4

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

ultralytics.utils.metrics.DetMetrics object with attributes:

ap_class_index: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

box: ultralytics.utils.metrics.Metric object

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x000002D1238F98E0>

curves: ['Precision Recall(B)', 'F1 Confidence(B)', 'Precision Confidence(B)', 'Recall Confidence(B)']

```
curves_results: [[array([
    0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005,
    0.006006, 0.007007, 0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013,
    0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021,
    0.022022, 0.023023,
    0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003,
    0.031031, 0.032032, 0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038,
    0.039039, 0.04004, 0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046,
    0.047047,
    0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053053, 0.054054,
    0.055055, 0.056056, 0.057057, 0.058058, 0.059059, 0.06006, 0.061061, 0.062062,
    0.063063, 0.064064, 0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007,
    0.071071,
    0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077077, 0.078078,
    0.079079, 0.08008, 0.081081, 0.082082, 0.083083, 0.084084, 0.085085, 0.086086,
    0.087087, 0.088088, 0.089089, 0.09009, 0.091091, 0.092092, 0.093093, 0.094094,
    0.095095,
    0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0.1011, 0.1021, 0.1031,
    0.1041, 0.10511, 0.10611, 0.10711, 0.10811, 0.10911, 0.11011, 0.11111,
    0.11211, 0.11311, 0.11411, 0.11512, 0.11612, 0.11712, 0.11812, 0.11912,
```

0.12012, 0.12112, 0.12212, 0.12312, 0.12412, 0.12513, 0.12613, 0.12713,
0.12813, 0.12913, 0.13013, 0.13113, 0.13213, 0.13313, 0.13413, 0.13514,
0.13614, 0.13714, 0.13814, 0.13914, 0.14014, 0.14114, 0.14214, 0.14314,

0.14414, 0.14515, 0.14615, 0.14715, 0.14815, 0.14915, 0.15015, 0.15115,
0.15215, 0.15315, 0.15415, 0.15516, 0.15616, 0.15716, 0.15816, 0.15916,
0.16016, 0.16116, 0.16216, 0.16316, 0.16416, 0.16517, 0.16617, 0.16717,

0.16817, 0.16917, 0.17017, 0.17117, 0.17217, 0.17317, 0.17417, 0.17518,
0.17618, 0.17718, 0.17818, 0.17918, 0.18018, 0.18118, 0.18218, 0.18318,
0.18418, 0.18519, 0.18619, 0.18719, 0.18819, 0.18919, 0.19019, 0.19119,

0.19219, 0.19319, 0.19419, 0.1952, 0.1962, 0.1972, 0.1982, 0.1992,
0.2002, 0.2012, 0.2022, 0.2032, 0.2042, 0.20521, 0.20621, 0.20721,
0.20821, 0.20921, 0.21021, 0.21121, 0.21221, 0.21321, 0.21421, 0.21522,

0.21622, 0.21722, 0.21822, 0.21922, 0.22022, 0.22122, 0.22222, 0.22322,
0.22422, 0.22523, 0.22623, 0.22723, 0.22823, 0.22923, 0.23023, 0.23123,
0.23223, 0.23323, 0.23423, 0.23524, 0.23624, 0.23724, 0.23824, 0.23924,

0.24024, 0.24124, 0.24224, 0.24324, 0.24424, 0.24525, 0.24625, 0.24725,
0.24825, 0.24925, 0.25025, 0.25125, 0.25225, 0.25325, 0.25425, 0.25526,
0.25626, 0.25726, 0.25826, 0.25926, 0.26026, 0.26126, 0.26226, 0.26326,

0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.26927, 0.27027, 0.27127,
0.27227, 0.27327, 0.27427, 0.27528, 0.27628, 0.27728, 0.27828, 0.27928,
0.28028, 0.28128, 0.28228, 0.28328, 0.28428, 0.28529, 0.28629, 0.28729,

0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.29329, 0.29429, 0.2953,
0.2963, 0.2973, 0.2983, 0.2993, 0.3003, 0.3013, 0.3023, 0.3033, 0.3043,
0.30531, 0.30631, 0.30731, 0.30831, 0.30931, 0.31031, 0.31131,

0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.31732, 0.31832, 0.31932,
 0.32032, 0.32132, 0.32232, 0.32332, 0.32432, 0.32533, 0.32633, 0.32733,
 0.32833, 0.32933, 0.33033, 0.33133, 0.33233, 0.33333, 0.33433, 0.33534,

0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.34134, 0.34234, 0.34334,
 0.34434, 0.34535, 0.34635, 0.34735, 0.34835, 0.34935, 0.35035, 0.35135,
 0.35235, 0.35335, 0.35435, 0.35536, 0.35636, 0.35736, 0.35836, 0.35936,

0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36537, 0.36637, 0.36737,
 0.36837, 0.36937, 0.37037, 0.37137, 0.37237, 0.37337, 0.37437, 0.37538,
 0.37638, 0.37738, 0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,

0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.38939, 0.39039, 0.39139,
 0.39239, 0.39339, 0.39439, 0.3954, 0.3964, 0.3974, 0.3984, 0.3994,
 0.4004, 0.4014, 0.4024, 0.4034, 0.4044, 0.40541, 0.40641, 0.40741,

0.40841, 0.40941, 0.41041, 0.41141, 0.41241, 0.41341, 0.41441, 0.41542,
 0.41642, 0.41742, 0.41842, 0.41942, 0.42042, 0.42142, 0.42242, 0.42342,
 0.42442, 0.42543, 0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,

0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43744, 0.43844, 0.43944,
 0.44044, 0.44144, 0.44244, 0.44344, 0.44444, 0.44545, 0.44645, 0.44745,
 0.44845, 0.44945, 0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,

...

results_dict: {'metrics/precision(B)': 0.35488380770452116, 'metrics/recall(B)':
 0.27278230032967077, 'metrics/mAP50(B)': 0.25480646478572005, 'metrics/mAP50 95(B)':
 0.14556773038216042, 'fitness': 0.14556773038216042}

save_dir: WindowsPath('C:/Users/these/Desktop/FYP2025
 SeanMaloney/runs/detect/visdrone_parking_detector4')

speed: {'preprocess': 0.13904233576523192, 'inference': 1.2396642335719343, 'loss':
 0.0004197080351909823, 'postprocess': 2.5497244525341842}

stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}

task: 'detect'

Ultralytics 8.4.11 Python 3.12.10 torch 2.11.0.dev20260203+cu128 CUDA:0 (NVIDIA GeForce RTX 5070 Laptop GPU, 8151MiB)

Model summary (fused): 73 layers, 3,007,598 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.00.0 ms, read: 914.8379.0 MB/s, size: 126.7 KB)

val: Scanning [C:\Users\these\Desktop\FYP2025](#)

[SeanMaloney\YOLOv8\datasets\VisDrone\labels\val.cache...](#) 548 images, 0 backgrounds, 0

corrupt: 100%  548/548 0.0s

Class Images Instances Box(P R mAP50 mAP50 95): 100% 

 35/35 6.6it/s 5.3s0.1s

all	548	38759	0.354	0.273	0.255	0.146
pedestrian	520	8844	0.337	0.31	0.272	0.113
people	482	5125	0.393	0.223	0.213	0.0712
bicycle	364	1287	0.166	0.0458	0.0409	0.013
car	515	14064	0.508	0.725	0.688	0.452
van	421	1975	0.392	0.275	0.278	0.189
truck	266	750	0.372	0.229	0.23	0.154
tricycle	337	1045	0.351	0.169	0.158	0.0866

awning tricycle	220	532	0.199	0.105	0.0755	0.048
bus	131	251	0.483	0.323	0.343	0.239
motor	485	4886	0.341	0.323	0.252	0.0927

Speed: 0.5ms preprocess, 3.0ms inference, 0.0ms loss, 2.2ms postprocess per image

Results saved to <C:\Users\these\Desktop\FYP2025 SeanMaloney\runs\detect\val>

ultralytics.utils.metrics.DetMetrics object with attributes:

ap_class_index: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

box: ultralytics.utils.metrics.Metric object

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x000002D157642780>

curves: ['Precision Recall(B)', 'F1 Confidence(B)', 'Precision Confidence(B)', 'Recall Confidence(B)']

curves_results: [[array([0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021, 0.022022, 0.023023, 0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047, 0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053053, 0.054054, 0.055055, 0.056056, 0.057057, 0.058058, 0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065, 0.066066, 0.067067, 0.068068, 0.069069, 0.07007, 0.071071,

0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077077, 0.078078,
0.079079, 0.08008, 0.081081, 0.082082, 0.083083, 0.084084, 0.085085, 0.086086,
0.087087, 0.088088, 0.089089, 0.09009, 0.091091, 0.092092, 0.093093, 0.094094,
0.095095,

0.096096, 0.097097, 0.098098, 0.099099, 0.1001, 0.1011, 0.1021, 0.1031,
0.1041, 0.10511, 0.10611, 0.10711, 0.10811, 0.10911, 0.11011, 0.11111,
0.11211, 0.11311, 0.11411, 0.11512, 0.11612, 0.11712, 0.11812, 0.11912,

0.12012, 0.12112, 0.12212, 0.12312, 0.12412, 0.12513, 0.12613, 0.12713,
0.12813, 0.12913, 0.13013, 0.13113, 0.13213, 0.13313, 0.13413, 0.13514,
0.13614, 0.13714, 0.13814, 0.13914, 0.14014, 0.14114, 0.14214, 0.14314,

0.14414, 0.14515, 0.14615, 0.14715, 0.14815, 0.14915, 0.15015, 0.15115,
0.15215, 0.15315, 0.15415, 0.15516, 0.15616, 0.15716, 0.15816, 0.15916,
0.16016, 0.16116, 0.16216, 0.16316, 0.16416, 0.16517, 0.16617, 0.16717,

0.16817, 0.16917, 0.17017, 0.17117, 0.17217, 0.17317, 0.17417, 0.17518,
0.17618, 0.17718, 0.17818, 0.17918, 0.18018, 0.18118, 0.18218, 0.18318,
0.18418, 0.18519, 0.18619, 0.18719, 0.18819, 0.18919, 0.19019, 0.19119,

0.19219, 0.19319, 0.19419, 0.1952, 0.1962, 0.1972, 0.1982, 0.1992,
0.2002, 0.2012, 0.2022, 0.2032, 0.2042, 0.20521, 0.20621, 0.20721,
0.20821, 0.20921, 0.21021, 0.21121, 0.21221, 0.21321, 0.21421, 0.21522,

0.21622, 0.21722, 0.21822, 0.21922, 0.22022, 0.22122, 0.22222, 0.22322,
0.22422, 0.22523, 0.22623, 0.22723, 0.22823, 0.22923, 0.23023, 0.23123,
0.23223, 0.23323, 0.23423, 0.23524, 0.23624, 0.23724, 0.23824, 0.23924,

0.24024, 0.24124, 0.24224, 0.24324, 0.24424, 0.24525, 0.24625, 0.24725,
0.24825, 0.24925, 0.25025, 0.25125, 0.25225, 0.25325, 0.25425, 0.25526,
0.25626, 0.25726, 0.25826, 0.25926, 0.26026, 0.26126, 0.26226, 0.26326,

0.26426, 0.26527, 0.26627, 0.26727, 0.26827, 0.26927, 0.27027, 0.27127,
0.27227, 0.27327, 0.27427, 0.27528, 0.27628, 0.27728, 0.27828, 0.27928,
0.28028, 0.28128, 0.28228, 0.28328, 0.28428, 0.28529, 0.28629, 0.28729,

0.28829, 0.28929, 0.29029, 0.29129, 0.29229, 0.29329, 0.29429, 0.2953,
0.2963, 0.2973, 0.2983, 0.2993, 0.3003, 0.3013, 0.3023, 0.3033, 0.3043,
0.30531, 0.30631, 0.30731, 0.30831, 0.30931, 0.31031, 0.31131,

0.31231, 0.31331, 0.31431, 0.31532, 0.31632, 0.31732, 0.31832, 0.31932,
0.32032, 0.32132, 0.32232, 0.32332, 0.32432, 0.32533, 0.32633, 0.32733,
0.32833, 0.32933, 0.33033, 0.33133, 0.33233, 0.33333, 0.33433, 0.33534,

0.33634, 0.33734, 0.33834, 0.33934, 0.34034, 0.34134, 0.34234, 0.34334,
0.34434, 0.34535, 0.34635, 0.34735, 0.34835, 0.34935, 0.35035, 0.35135,
0.35235, 0.35335, 0.35435, 0.35536, 0.35636, 0.35736, 0.35836, 0.35936,

0.36036, 0.36136, 0.36236, 0.36336, 0.36436, 0.36537, 0.36637, 0.36737,
0.36837, 0.36937, 0.37037, 0.37137, 0.37237, 0.37337, 0.37437, 0.37538,
0.37638, 0.37738, 0.37838, 0.37938, 0.38038, 0.38138, 0.38238, 0.38338,

0.38438, 0.38539, 0.38639, 0.38739, 0.38839, 0.38939, 0.39039, 0.39139,
0.39239, 0.39339, 0.39439, 0.3954, 0.3964, 0.3974, 0.3984, 0.3994,
0.4004, 0.4014, 0.4024, 0.4034, 0.4044, 0.40541, 0.40641, 0.40741,

0.40841, 0.40941, 0.41041, 0.41141, 0.41241, 0.41341, 0.41441, 0.41542,
0.41642, 0.41742, 0.41842, 0.41942, 0.42042, 0.42142, 0.42242, 0.42342,
0.42442, 0.42543, 0.42643, 0.42743, 0.42843, 0.42943, 0.43043, 0.43143,

0.43243, 0.43343, 0.43443, 0.43544, 0.43644, 0.43744, 0.43844, 0.43944,
0.44044, 0.44144, 0.44244, 0.44344, 0.44444, 0.44545, 0.44645, 0.44745,
0.44845, 0.44945, 0.45045, 0.45145, 0.45245, 0.45345, 0.45445, 0.45546,

...


```

results_dict: {'metrics/precision(B)': 0.35398661429164247, 'metrics/recall(B)':
0.272828997715951, 'metrics/mAP50(B)': 0.25509012908983986, 'metrics/mAP50 95(B)':
0.14579528209909057, 'fitness': 0.14579528209909057}

save_dir: WindowsPath('C:/Users/these/Desktop/FYP2025 SeanMaloney/runs/detect/val')

speed: {'preprocess': 0.5112899634961624, 'inference': 3.035937226266562, 'loss':
0.00028686132069581947, 'postprocess': 2.199123175196424}

stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}

task: 'detect'

```

WARNING

Inference results will accumulate in RAM unless `stream=True` is passed, which can cause out of memory errors for large

sources or long running streams and videos. See <https://docs.ultralytics.com/modes/predict/> for help.

Example:

```

results = model(source=..., stream=True) # generator of Results objects

for r in results:

    boxes = r.boxes # Boxes object for bbox outputs

```

masks = r.masks # Masks object for segment masks outputs

probs = r.probs # Class probabilities for classification outputs

video 1/1 (frame 1/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 5 pedestrians, 209 cars, 10 vans, 46.2ms

video 1/1 (frame 2/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 3 pedestrians, 218 cars, 13 vans, 8.8ms

video 1/1 (frame 3/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 222 cars, 13 vans, 8.5ms

video 1/1 (frame 4/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 3 pedestrians, 221 cars, 13 vans, 8.9ms

video 1/1 (frame 5/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 224 cars, 14 vans, 8.5ms

video 1/1 (frame 6/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 3 pedestrians, 222 cars, 13 vans, 8.5ms

video 1/1 (frame 7/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 226 cars, 12 vans, 7.5ms

video 1/1 (frame 8/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 224 cars, 15 vans, 8.5ms

video 1/1 (frame 9/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 229 cars, 16 vans, 9.7ms

video 1/1 (frame 10/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 226 cars, 13 vans, 13.9ms

video 1/1 (frame 11/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 225 cars, 11 vans, 8.3ms

video 1/1 (frame 12/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 226 cars, 11 vans, 11.2ms

video 1/1 (frame 13/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 1 pedestrian, 223 cars, 12 vans, 9.2ms

...

video 1/1 (frame 372/376) <c:\Users\these\Desktop\FYP2025>

<SeanMaloney\YOLOv8\Assets\Video1.mp4>: 384x640 2 pedestrians, 224 cars, 8 vans, 1 truck,
11.1ms

Speed: 2.0ms preprocess, 11.8ms inference, 2.0ms postprocess per image at shape (1, 3, 384, 640)

Results saved to <C:\Users\these\Desktop\FYP2025>

SeanMaloney\runs\detect\runs\detect\parking_output

 Inference complete. Output saved to: <C:\Users\these\Desktop\FYP2025>

SeanMaloney\runs\detect\runs\detect\parking_output